Separation of Concerns at Zero Cost

Modular Physics in C++20

Jędrzej Michalczyk

BEFORE WE START

About me

- **Jędrzej Michalczyk** — R&D Engineer
- **SpaceForest** (Gdynia, Poland)
  - PERUN — single-stage suborbital launch vehicle
  - guidance, navigation & control software in modern C++
- The code in this talk is roughly : github.com/jedrzejmichalczyk/accu-on-sea-2026-modular-physics

The original problem

- Simulate a rocket: **6-DOF dynamics + aerodynamics + propulsion + atmosphere + control**
- The same physics must run in three places:
  - engineering analysis (parameter studies, Monte Carlo)
  - hardware-in-the-loop test benches — real time
  - onboard — guidance runs against the same models, in real time
- **Correctness and performance are crucial**
- And: aero engineers, control engineers and software engineers all touch the same code

THE GENERAL FLOW

From symbolic math to embedded C

1. High-level framework

MATLAB · Mathematica · Symbolica

Express the equations once and validate the physics at a high level.

2. Rewrite or generate

Hand-rewrite or auto-generate

Translate the validated model into portable C from the symbolic source.

3. Embedded C

Compile & deploy

Runs on the target — fast, deterministic and real-time.

Sometimes it is an overkill.

01

The Principle

Separation of concerns in numerical code

≈ 5 min

THE MODELING PROBLEM

Where do the software modules go?

It is not obvious where to draw the software module boundaries.

THE PRINCIPLE

Choose the states — everything else is a state function

SOPOT — “Obviously Physics, Obviously Templates” — tries to capture this idea

THE STATUS QUO

What numerical code usually looks like

```cpp
// derivs.cpp — all of physics in one function
void derivs(double t, const double* y, double* dydt) {
    // y[0..2] pos, y[3..5] vel, y[6..9] quat, y[10..12] omega, y[13] mass
    double rho = 1.225 * exp(-y[2] / 8500.0);              // atmosphere
    double v2  = y[3]*y[3] + y[4]*y[4] + y[5]*y[5];
    double Fd  = 0.5 * rho * v2 * CD * AREA;               // aero drag
    double Ft  = thrust_table_lookup(t);                   // propulsion
    dydt[3] = (Ft*ax(y) - Fd*vx_hat(y)) / y[13];           // dynamics
    /* ... 400 more lines, every subsystem interleaved ... */
}
```

02

The Tradeoff That Isn't

Modularity vs performance — or so we're told

≈ 7 min

THE TEXTBOOK FIX

Attempt: classic OO interfaces

```
struct Component {
    virtual void computeDerivatives(double t,
                                    const double* state,
                                    double* dydt) = 0;
    virtual double getOutput(const std::string& name) = 0;  // !!
    virtual ~Component() = default;
};

std::vector<std::unique_ptr<Component>> components;
for (auto& c : components)                // innermost loop of RK4,
    c->computeDerivatives(t, y, dydt);  // 4x per step, 1000s of steps
```

PICK ONE?

The apparent tradeoff

Monolith	Virtual components
fast ✓	modular ✓
unmaintainable ✗	indirection in the hot loop ✗

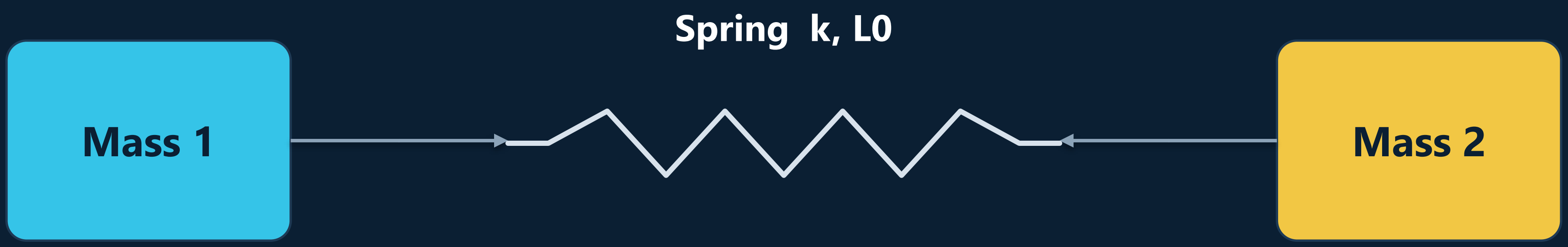
Can be done

03

The Coupled Oscillator

Two masses, one spring, four components

≈ 8 min



EnergyMonitor

$KE_1 + KE_2 + PE = \text{const?}$

SEPARATION, CONCRETELY

Who owns what — who asks what

Component	Owns (state)	Provides	Queries
Mass1 / Mass2	[position, velocity]	Position, Velocity, Mass	Force (whoever provides it)
Spring	— nothing	Force, Extension, PE	Position of both masses
EnergyMonitor	— nothing	TotalEnergy, Momentum	everything above

- Composition of the whole system
- Wiring
-

04

Code Walkthrough

The four components, for real

≈ 12 min

WALKTHROUGH 1/5

Tags: typed names for physical quantities

```cpp
struct mass1 {
    struct Position {};      // empty types — pure compile-time keys
    struct Velocity {};
    struct Force    {};
    struct Mass     {};
};

struct mass2 { /* same shape, different types */ };

namespace spring {
    struct Extension       {};
    struct PotentialEnergy {};
};
```

WALKTHROUGH 2/5

PointMass: owns its state, asks for force

```cpp
template<typename TagSet, Scalar T = double>
class PointMass final : public Component<2, T> {   // 2 states: [x, v]
    double m_mass;
public:
    template<typename Registry>
    LocalDerivative derivatives(T t, std::span<const T> state, const Registry& registry) const {
        T velocity = this->localState(state, 1);          // dx/dt = v
        T force = query<typename TagSet::Force>(registry, state);
        return {velocity, force / T(m_mass)};             // dv/dt = F/m
    }
    template<typename Registry>                // Velocity: same, localState(s, 1)
    T compute(typename TagSet::Position, std::span<const T> s,
              const Registry&) const { return this->localState(s, 0); }
};
```

Spring: no state, just physics

```cpp
template<typename TagSet1, typename TagSet2, Scalar T = double>
class Spring final : public Component<0, T> {        // 0 states!
    double m_stiffness, m_rest_length;
public:
    template<typename Registry>
    T compute(typename TagSet1::Force, std::span<const T> state, const Registry& reg) const {
        T x1 = query<typename TagSet1::Position>(reg, state);
        T x2 = query<typename TagSet2::Position>(reg, state);
        T extension = x1 - x2 - T(m_rest_length);
        return -T(m_stiffness) * extension;               // Hooke's law
    }
    template<typename Registry>
    T compute(typename TagSet2::Force, std::span<const T> state, const Registry& reg) const {
        return -query<typename TagSet1::Force>(reg, state);      // Newton's 3rd
    }
};
```

WALKTHROUGH 4/5

EnergyMonitor: system totals from component data

```
template<Scalar T = double>
class EnergyMonitor final : public Component<0, T> {
public:
    template<typename Registry>
    T compute(system::TotalEnergy,
              std::span<const T> state, const Registry& registry) const {
        T m1 = query<mass1::Mass>(registry, state);
        T v1 = query<mass1::Velocity>(registry, state);
        T m2 = query<mass2::Mass>(registry, state);
        T v2 = query<mass2::Velocity>(registry, state);
        T pe = query<spring::PotentialEnergy>(registry, state);
        return T(0.5)*m1*v1*v1 + T(0.5)*m2*v2*v2 + pe;
    }
};
```

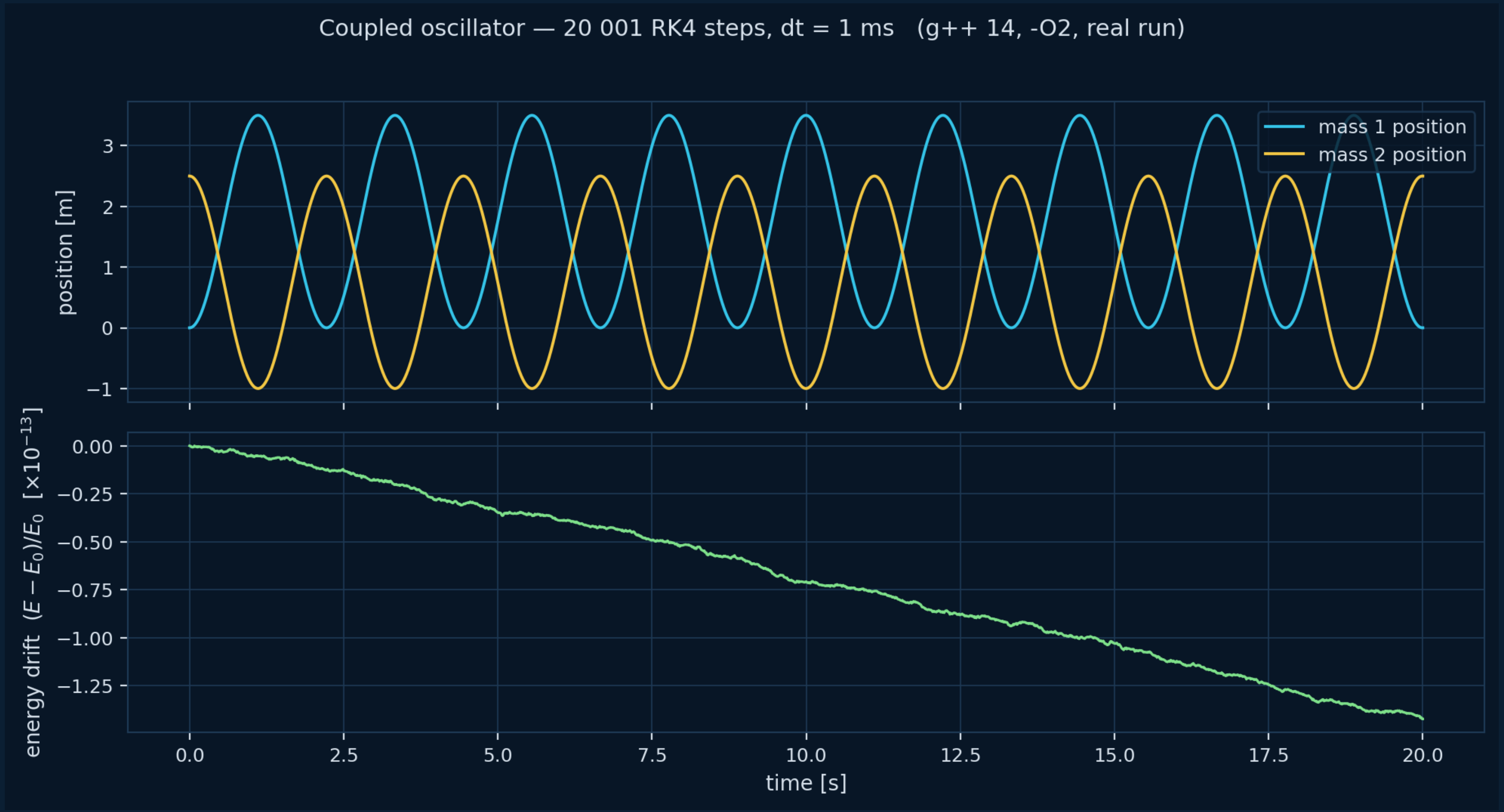
WALKTHROUGH 5/5

Wiring it together

```cpp
auto sys = makeODESystem<double>(
    Mass1<double>(1.0, /*x0*/ 0.0, /*v0*/ 0.0),
    Mass2<double>(1.0, /*x0*/ 2.5, /*v0*/ 0.0),
    Spring12<double>(/*k*/ 4.0, /*L0*/ 1.0),
    EnergyMonitor<double>()
);

// the wiring is CHECKED at compile time:
static_assert(decltype(sys)::hasFunction<system::TotalEnergy>());

RK4Solver solver;
auto sol = solver.solve(derivs(sys), sys.getStateDimension(),
                        0.0, 20.0, /*dt*/ 0.001, sys.getInitialState());

double E = sys.query<system::TotalEnergy>(sol.states.back());
```

PROOF, NOT PROMISE

Does it work?



20 001

RK4 steps (dt = 1 ms)

16.7 ms

wall clock, g++ 14 -O2

1.4×10^{-13}

max relative energy drift over 20 s

05

How It Works

Tags, registry, concepts — no magic

≈ 10 min

Idea 1 — dispatch on type, not through a hierarchy

```
// instead of:
double getOutput(
    const std::string& name);

// one virtual + string compare
// per query, at runtime
```

```
// we write:
T compute(mass1::Position,
    std::span<const T> state) const;

// one overload per quantity,
// chosen by the COMPILER
```

- **Overload resolution replaces virtual dispatch**
- The tag argument is empty — it exists only to select the overload
- Unknown tag → compile error, with the tag's name in the message
- This is ordinary C++ — tag dispatch is as old as the STL (std::advance)

HOW IT WORKS 2/5

Idea 2 — the registry: 'who provides what'

```
template<typename T, ComponentConcept... Components>
class Registry {
    std::tuple<const Components&...> m_components;

    template<StateTagConcept Tag>
    static constexpr bool hasFunction() {
        return (ProvidesStateFunction<Components, Tag, T, Registry> || ...);
    }                                     //         fold over all components

    template<StateTagConcept Tag>
    auto computeFunction(std::span<const T> state) const {
        static_assert(hasFunction<Tag>(),
                      "No component provides this state function");
        const auto& provider = findProvider<Tag>();   // index found constexpr
        return provider.compute(Tag{}, state, *this);
    }
};
```

HOW IT WORKS 3/5

compute vs query: provide vs ask

```
// a component PROVIDES a quantity (one overload per tag):
T compute(mass1::Force, std::span<const T> s, const Registry&) const;

// anyone ASKS for one, by tag — component or driver:
T f = query<mass1::Force>(registry, state);

// query is just the registry call with friendlier syntax:
template<StateTagConcept Tag, typename Registry, typename T>
auto query(const Registry& reg, std::span<const T> s) {
    return reg.computeFunction<Tag>(s);   // find provider -> provider.compute(...)
}
```

HOW IT WORKS 4/5

Idea 3 — concepts make the contracts checkable

```
// what a component IS:
template<typename C>
concept ComponentConcept = requires(const C& c) {
    { C::state_size } -> std::convertible_to<size_t>;
    { c.getInitialLocalState() } -> std::same_as<typename C::LocalState>;
};

// what it PROVIDES (a quantity, selected by tag):
template<typename C, typename Tag, typename T, typename Registry>
concept ProvidesStateFunction =
    ComponentConcept<C> && StateTagConcept<Tag> &&
    requires(const C& c, std::span<const T> s, const Registry& reg)
    { c.compute(Tag{}, s, reg); };
```

Idea 4 — everything is a value

- The system is std::tuple<Mass1, Mass2, Spring, EnergyMonitor>
  - no heap, no pointers, no type erasure, no std::function
- **The optimizer sees the whole object graph as one struct**
  - every compute() call is a direct (and inlinable) call
- State lives in one contiguous vector; components know their offsets
- The price: component set is fixed at compile time
  - for flight software that's a feature — the vehicle doesn't grow a fin mid-flight

06

What the Compiler Sees

Trust, but disassemble

≈ 6 min

G++ 14 -O2, REAL OUTPUT

The abstraction compiles away

```
energy_sopot(span<const double>):      ; 4 components, registry dispatch
    mov   r10, [sys+272]
    movsd xmm1, [.LC0]          ; 0.5
    mov   rax, [rcx]
    movsd xmm0, [r10+8]         ; m1        <- runtime parameters,
    movsd xmm3, [rax+rcx*8]     ; v1           loaded once
    mulsd xmm0, xmm1
    mulsd xmm0, xmm3            ; 0.5*m1*v1
    mulsd xmm0, xmm3            ; ... * v1
    subsd xmm2, [r8+16]         ; x1-x2-L0
    mulsd xmm1, xmm2            ; PE terms
    addsd xmm0, xmm3
    addsd xmm0, xmm1            ; KE1+KE2+PE
    ret
```

0 calls · 0 jumps · 0 vtables — straight-line FP, the same shape as the hand-written formula.

…and the derivative? Same story.

```
mass1_derivative(...):              ; returns { dx/dt , dv/dt }
    mov   r9,  [sys+112]            ; spring component
    mov   r10, [sys+128]            ; mass1 component
    movsd xmm2, [r9+8]              ; stiffness k, negated -> -k
    xorpd xmm2, [.LC0]
    movsd xmm0, [r8+rcx*8]          ; x1
    subsd xmm0, [r8+rdx*8]          ;   - x2
    subsd xmm0, [r9+16]             ;   - L0       -> extension
    movsd xmm1, [r8+rcx*8]          ; v1           (= dx/dt)
    mulsd xmm0, xmm2                ; -k * extension   (spring force)
    comisd xmm3, 0                 ; if (damping > 0)  - c == 0, so
    jbe   .Lskip                   ;   the branch is skipped
    divsd xmm0, [r10+8]            ; force / m1   (= dv/dt)
    unpcklpd xmm1, xmm0            ; pack { v1, a1 }
    movups [rax], xmm1
    ret
```

the whole query chain — spring force, then ÷ mass — is ~15 instructions, no calls.

MEASURED, NOT ESTIMATED

Numbers

~1 ns per state-function query — fully inlined

16.7 ms 20 001 RK4 steps of the 4-component oscillator (laptop, -O2)

1000× faster than an equivalent OO C# version

FROM THE HANDWRITTEN PLAN

Why C++20 — and not C++26?

- **Everything here needs only C++20:**
  - concepts (the diagnostics!), fold expressions, std::span, CTAD
- **C++26 reflection (P2996) would remove the remaining boilerplate:**
  - tag declarations and compute() registration could be generated
  - we have working experiments in the repo: reflect/
- But flight software ships on the toolchain you have, **not the one you want**
  - embedded / cross targets lag years behind; certification lags more

07

Where It Scales

From two masses to a flying vehicle

≈ 7 min

Same pattern, real vehicle: 6-DOF rocket

- **11 components, 14 states** — position, velocity, quaternion, angular rate, mass
- Thrust vector control & attitude determination: same tags-and-registry architecture
- **These patterns run in production flight software** — correctness and performance both non-negotiable

Bonus: the scalar is a template parameter

```
// simulate:
auto sim  = makeODESystem<double>(comps...);

// differentiate — SAME components, different scalar:
using D = Dual<double, 14>;                      // forward-mode autodiff
auto grad = makeODESystem<D>(comps...);
auto J    = computeJacobian(grad, t, state); // exact, to machine precision

// linearize -> LQR gain straight from the nonlinear model
auto K = lqr(J.A, J.B, Q, R);
```

- **Jacobians without hand-derived math or finite differences**
- The controller you'll see in a minute was designed exactly this way

OPEN IDEAS

Can we go further?

- **Frames as types** — body vs ENU vs ECEF: mixing them up should be a compile error too
- **Physical laws as concepts** — 'conserves energy', 'action-reaction pair' as checkable contracts
- Units in the type system — dimensional analysis at compile time

The goal, all the way down: a consistent level of abstraction

A SHORT HISTORY LESSON

We've been here before

```
XISP1=250.;  XISP2=250.;  XMF1=.85;  XMF2=.85;
WPAY=100.;   DELV=20000.;
DELV1=.3333*DELV;   DELV2=.6667*DELV;
AMAX1=10.;   AMAX2=10.;   GAMDEG=85.;
TOP2=WPAY*(exp(DELV2/(XISP2*32.2))-1.);
BOT2=1/XMF2-((1.-XMF2)/XMF2)*exp(DELV2/(XISP2*32.2));
WP2=TOP2/BOT2;   WS2=WP2*(1-XMF2)/XMF2;   WTOT2=WP2+WS2+WPAY;
TRST2=AMAX2*(WPAY+WS2);   TB2=XISP2*WP2/TRST2;
% ... then TOP1,BOT1,WP1,WS1,WTOT,TRST1,TB1 -- same again for stage 1
```

Zarchan, Tactical & Strategic Missile Guidance, 6th ed. (MATLAB) — gravity-turn booster sim

- Six decades of rocket code — the math was always modular; the code never was
- **Not because the engineers were careless — because the languages made them choose**

A SHORT HISTORY LESSON

One loop, every variable global

```
while ~(ALT<0 & T>10)
    if T<TB1
        WGT=-WP1*T/TB1+WTOT;   TRST=TRST1;
    elseif T<(TB1+TB2)
        WGT=-WP2*T/TB2+WTOT2+WP2*TB1/TB2;   TRST=TRST2;
    else
        WGT=WPAY;   TRST=0.;
    end
    AT=32.2*TRST/WGT;   VEL=sqrt(X1^2+Y1^2);
    TEMBOT=(X^2+Y^2)^1.5;
    X1D=-GM*X/TEMBOT+AT*X1/VEL;
    Y1D=-GM*Y/TEMBOT+AT*Y1/VEL;
    ...                         % + Euler step, range/altitude bookkeeping
end
```

Zarchan, Tactical & Strategic Missile Guidance, 6th ed. (MATLAB) — gravity-turn booster sim

One scope. Every name global. Physics, staging, and the integrator all tangled together.

A SHORT HISTORY LESSON

Same script — even the plotting and file I/O

```
    if S>=9.99999
        R=sqrt(X^2+Y^2);   RF=sqrt(XFIRST^2+YFIRST^2);
        BETA=acos((X*XFIRST+Y*YFIRST)/(R*RF));
        DISTNM=A*BETA/6076.;   ALTNM=(sqrt(X^2+Y^2)-A)/6076.;
        count=count+1;
        ArrayT(count)=T;  ArrayDISTNM(count)=DISTNM;  ArrayALTNM(count)=ALTNM;
    end
end
figure
plot(ArrayDISTNM,ArrayALTNM),grid
xlabel('Downrange (Nmi)');   ylabel('Altitude (Nmi)');
save datfil output /ascii
disp 'simulation finished'
```

Zarchan, Tactical & Strategic Missile Guidance, 6th ed. (MATLAB) — gravity-turn booster sim

No boundary between simulation, analysis, and I/O — every concern in one scope.

IF YOU REMEMBER FOUR THINGS

Takeaways

- **Separation of concerns is achievable at zero cost** in C++20 — measured, not asserted
- The recipe is small: **typed tags + overloaded compute() + a constexpr registry + concepts**
  - no inheritance, no virtuals, no strings, no codegen
- **Prefer abstractions you can verify** — conservation laws and disassembly are unit tests for architecture
- **Quality vs performance was the false tradeoff** — modern C++ finally lets us refuse it

Thank you! Questions?

- Code & slides: **github.com/jedrzejmichalczyk/accu-on-sea-2026-modular-physics**
- Email: **jedrzej.michalczyk@gmail.com**